

Lab assignment 1: Debugging and version control

Introduction to programming (LIS020)

Robert Östling Amanda Kann
robert@ling.su.se amanda.kann@ling.su.se

July 10, 2026

Purpose and learning outcomes

In this assignment, you will work with pre-existing code for generating strings from formal grammars, such as the context-free grammar used as an example in the lab. However, the code contains a number of problems which you need to identify with the help of automated code checking tools. You will also need to use your own brain to find logic errors of the kind that automated tools have problems identifying.

To achieve a given grade for this assignment, you will need to:

- **Pass:** Understand the code for working with formal grammars, and correct the problems it contains so that the built-in demo works correctly.
- **Pass with distinction:** Use the `git` version management system to track and document the changes you make to the code. Since the code contains about half a dozen errors, we expect the corresponding number of commits – one for each error. Each commit message should be a one-line description describing what type of error you fixed.

Submitting your solution

Which file to submit depends on which grade you are aiming for:

- **Pass:** Submit the file `grammar.py` to Canvas.
- **Pass with distinction:** Create a compressed tar archive of your git repository as follows:

```
tar cvzf grammar.tgz grammar/.git grammar/grammar.py
```

Then submit the file `grammar.tgz` to Canvas.

If you are working on the **triton** server, you can copy files from there to your local computer by executing the following command on **your** computer (not while logged into triton):

```
scp abcd1234@triton.dsv.su.se:lis020/lab1/grammar.tgz ./
```

Background

Errors in program code are inevitable, and even with decades of experience you will continue making them. There are many methods for finding errors: linters/static analysis, debuggers, asking LLMs, and simply staring at the code.

In this assignment you will practice using three tools for different types of automatic code analysis: **pylint**, **mypy** and **pycodestyle**. The types of problems they report have some overlap, but generally speaking **pylint** focuses on simple errors (misspelled variables, missing parentheses, etc.), **mypy** on type errors (e.g. using an integer when a list would be expected) and **pycodestyle** on style issues that do not impede functionality but make the code more difficult to read.

Tasks

Goal: pass (without distinction)

Start by giving the following command to copy the lab template to the directory of this assignment:

```
mkdir -p ~/lis020/lab1
cd ~/lis020/lab1
cp /home/dsv/robe/lis020/labs/grammar/grammar.py ./
```

Now, your goal is to change this file until it works as intended. Jump to the section *Common tasks* below for detailed instructions.

Goal: pass with distinction

Start by giving the following command to clone the git repository to your home directory:

```
mkdir -p ~/lis020/lab1
cd ~/lis020/lab1
git config --global --add safe.directory \
    /home/dsv/robe/lis020/labs/grammar/grammar.git
git clone /home/dsv/robe/lis020/labs/grammar/grammar.git
```

The `git config` command is needed in order to make recent versions of git accept the fact that you are cloning a repository from another user.

When you have fixed one error in the code, you should make a git commit and write a short one-line summary of what you did. When you run `'git commit -a'`, git will launch a text editor for you to write your commit message in. By default, this editor is `'nano'`. Simply write your message on the first line in the file (no hash sign needed, as in the default comments below), press `ctrl+X` to exit, then press `y` (for "yes", you want to save changes), then press `enter` to accept the default filename suggested.

Alternatively, you can add your message directly to the command, with the `'-m'` flag:

```
git commit -a -m "Fixed a syntax error"
```

Remember to put your message within quotation marks, otherwise the operating system will think all words after the first are arguments to `'git'`, and you will get confusing error messages!

You do not need to push your changes anywhere, the cloned git repository (with your changes) will *be* your solution to this lab assignment. The git repository consists of a large number of small files, which is why we ask you to create a zip file.

Common tasks

Your first task is to read the code and documentation contained in `grammar.py`, so that you understand what the code is supposed to do and roughly how the programmer intended to solve it. Next, you should edit the file `grammar.py` to run, and correctly print the only string that the example grammar in it can generate: "snails eat snails".

Automated tools can easily find syntactic errors for you, most of the incorrect variables, most type errors (especially if the code uses type annotations), but logic errors are much more difficult and you will typically need to read through the code and comments and figure these out yourself. Recently, LLMs such as ChatGPT and several code-specific ones, have become useful tools for finding these kinds of errors as well. They can not solve all cases however, and if the documentation is not detailed enough, only you know what you *want* your code to do, and then only you can fix the code when it does not align with your expectations. For this reason, **do not use LLMs** for this lab assignment, but try hard to solve the problems on your own. If you get stuck, you are allowed to discuss with the teachers or your classmates, but at the end of the day, you still need to understand how the code works. Remember, the goal of this assignment (and the entire course) is to make you a programmer, and programmers spend a huge amount of time figuring out why code does not work.

If you are curious about how well LLMs can solve this task, you are welcome to try using them **after** the lab deadline.

The setup of the `micro` editor used in this course includes automatic integration with `pylint`, `mypy` and `pycodestyle`, so the simplest way is to look for lines in the editor which begin with `>>`, move the cursor there and read the error message. You can also run these tools directly on the code from the terminal:

```
pylint grammar.py
```

and

```
mypy grammar.py
```

and

```
pycodestyle grammar.py
```

You can use whichever way you find most convenient. In case the error messages are too long to fit on a single line, `micro` will cut them off. To obtain the full message, run the corresponding tool as above.

Normally you proceed in the following order when fixing a faulty program:

1. Fix syntactic errors found by `pylint`. This comes first because the other tools often fail if there are syntactic errors present. In other words, once you fix the syntactic errors, other types of (previously undetected) errors often surface elsewhere in the code.
2. Fix type errors found by `mypy`. For instance, if you pass a list as an argument to a function that expects an integer, this is guaranteed to cause trouble sooner or later.
3. Have a serious look at other messages from `pylint`. These are of many different types, some more important than others. In some cases, warnings may lead you to actual errors, if the particular error happens to produce syntactically correct but weird code, that `pylint` warns about. These can be a bit tricky to fix, because the problem reported may be something completely unrelated to the actual problem. For instance, a warning about a variable that is defined but never used, often indicates that something is wrong in the code where you *intended* to use the variable.
4. At this stage the program should run. Test the code to ensure that it works as expected. If not, try to deduce (often with the help of further testing) why the result differs from your expectations. This is the way to find and fix logic errors.

5. Fix style issues and make a complete documentation.

Note that these tools have trouble judging the severity of the problems they report, and that errors that make the program not work at all are mixed with nitpicking about how the code is formatted visually. Try your best to follow the priority list above, and discuss with classmates or ask the teachers when in doubt.